

Predicción de la Edad de un Abulón a través de Machine Learning

Jorge Manuel Oyoqui Aguilera || A01711783

Inteligencia artificial avanzada para la ciencia de datos

Grupo 601

Profesor Benjamín Valdés Aguirre

30/08/2026

I. Introducción

1.1. Objetivo

El objetivo principal de mi modelo fue lograr predecir la edad de un abulón a través de sus características físicas implementando manualmente todo el proceso de aprendizaje de dicho modelo sin usar ningún framework o biblioteca de Machine Learning. Pero primero entendamos el problema, empezando por saber por qué nos interesaría conocer la edad de un abulón.

1.2. ¿Por qué predecir la edad de un abulón?

Compartiendo algo de contexto, los abulones son moluscos marinos en peligro de extinción que se encuentran en aguas costeras frías de todo el mundo, distribuidos frente a las costas de Nueva Zelanda, Sudáfrica, Australia, el oeste de Norteamérica y Japón. Son alimentos muy nutritivos, elegantes y poco comunes, y se consumen ampliamente en Francia, Nueva Zelanda, ciertas partes de Latinoamérica, Japón y Corea. Las conchas de abulón se utilizan con fines decorativos, por lo que el abulón tiene una gran importancia económica. El precio del abulón está directamente relacionado con su edad. Sin embargo, determinar la edad del abulón es un proceso muy difícil. Se forman anillos en la concha interna del abulón, uno por año. La edad del abulón se determina cortando la concha por la mitad, tiñéndola y contando el número de anillos con un microscopio.

Dado que algunos anillos son difíciles de detectar con este método, tradicionalmente se suma 1,5 al recuento de anillos como una aproximación razonable de la edad del abulón. Conocer el precio absoluto del abulón es importante tanto para los agricultores como para los consumidores, mientras que conocer la edad correcta es importante para los ambientalistas que buscan proteger esta especie en peligro de extinción.

Ahora, me gustaría aclarar que en mi dataset no había ninguna variable llamada Edad pero, como justamente ya mencioné, la forma más cercana de estimar la edad de un abulón es a través de su número de anillos usando la fórmula:

$$\text{Edad_abulón} = \text{Número_anillos} + 1.5$$

Así que el objetivo de mi modelo va a ser estimar la cantidad de anillos en un abulón a través de sus características físicas.

1.3. Modelo de Machine Learning a usar

Como el objetivo de mi modelo fue predecir el número de anillos (Rings) de un abulón, tuve que decidir con qué tipo de modelo iba a trabajar.

Recordemos que existen dos tipos de aprendizaje en Machine Learning: el aprendizaje supervisado y el no supervisado. En el caso del aprendizaje supervisado, los modelos se entrenan usando un conjunto de datos en el que la respuesta correcta ya se conoce. Con el aprendizaje no supervisado, el conjunto de datos que se usó para el entrenamiento el modelo no contiene la respuesta correcta.

Como en mi dataset si poseo el valor correcto de la cantidad de anillos de cada abulón, entonces eso significaba que trabajaría con un modelo de **algoritmo de aprendizaje supervisado**.

Este tipo de algoritmos de aprendizaje supervisado se dividen en dos tipos de problemas: de regresión y de clasificación. Los problemas de clasificación son aquellos donde el resultado esperado es una categoría o clase, mientras que en los problemas de regresión lo que se espera es la predicción de un valor continuo.

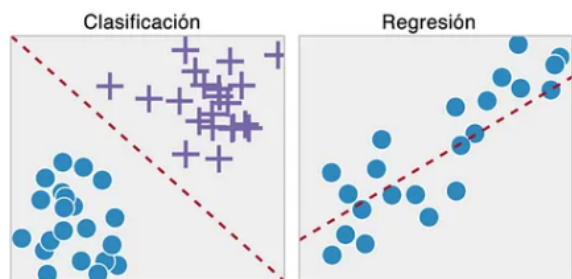


Figura 1. Algoritmos de clasificación y regresión

En mi caso, como voy a estar intentando predecir el valor de la variable Rings, es decir, un valor continuo, entonces el tipo de algoritmo que trabajaré para mi modelo fue uno de **regresión lineal múltiple** (múltiple debido a las múltiples variables independientes con las que trabajé).

Ahora, como no voy a poder utilizar frameworks, necesito un método de optimización para poder modificar y actualizar los pesos y el bias poco a poco para entrenar mi modelo y así reducir el MSE. Es ahí donde entra el Gradiente Descendente, con el cual decidí trabajar.

Así que, en general, con eso decidí que el modelo con el que iba a estar trabajando iba a ser un modelo de Regresión Lineal Múltiple con Gradiente Descendente.

Con eso claro, pasemos a la siguiente parte.

II. ETL

2.1. ¿Qué es ETL?

Cuando hablamos de Machine Learning, las siglas ETL significan extracción, transformación y carga de los datos. Este proceso es fundamental ya que es la fase para preparar, organizar y limpiar los datos que se usarán más adelante para entrenar y probar mi modelo. De esta forma, me aseguraré de tener datos limpios que el modelo pueda usar,

que no posea inconsistencias o sesgos y que se puedan usar para mi análisis.

2.2. Mi dataset

Para este proyecto, usé el dataset conocido como *abalone*, el cual se encuentra disponible en la siguiente liga: <https://archive.ics.uci.edu/dataset/1/abalone>.

El dataset proporciona la información de las características de los diferentes abulones que se estudiaron, las cuales serán usadas para predecir los valores de la variable *Rings*. Este dataset contiene en total 4177 instancias, cada una representando a un abulón diferente. Posee 9 características que describen las cualidades físicas de cada abulón. A continuación, explico más detalladamente qué representa cada variable:

Característica	Descripción
Sex	El el sexo del abulón. Se clasifica en M (male), F (female) e I (infant)
Length	Medida de la longitud máxima de la concha (en mm)
Diameter	Medida perpendicular a la longitud (en mm)
Height	La altura con la carne dentro de la concha (en mm)
Whole weight	Peso del abulón completo (en gramos)
Shucked weight	Peso de la carne sin la concha (en gramos)
Viscera weight	Peso del tracto digestivo tras el desangrado (en gramos)
Shell weight	Peso de la concha tras el secado (en gramos)
Rings	Número de anillos en la concha del abulón. +1.5 indica la edad en años del abulón (valor a predecir)

Tabla 1. Características de los abulones

2.3. Carga del dataset

Primero, descargué el dataset de UC Irvine Machine Learning Repository de la liga que antes compartí, para posteriormente usar la librería de pandas para cargar los datos en un dataframe llamado *df* y nombrar sus columnas según correspondiera con la variable cuyos datos iba a contener.

2.4. Limpieza de los datos

2.4.1. Detección de valores inválidos

Luego de cargar los datos en el dataset, empecé a revisar los datos hasta que me di cuenta de que el valor mínimo de la variable Height era exactamente de cero. Eso no debía ser correcto, así que decidí investigar más a fondo y buscar todas las instancias con un valor de Height = 0. Al final, obtuve dos resultados. Tenía la hipótesis de que se iban a tratar de instancias donde todos sus valores fueran de cero, pero resultó no ser así. Los únicos datos con cero eran sus alturas. Como esos datos iban a ser problemáticos, al ser solamente dos instancias de 4177, decidí mejor eliminarlas al tratarse de sólo un 0.048% del total de mis datos.

2.4.2. Detección de valores nulos

Revisé si mi dataset poseía alguna instancia con valores nulos. Afortunadamente, ninguna tenía valores nulos, así que ahí no tuve que hacer ninguna limpieza.

2.4.3. Detección de outliers

Lo siguiente fue buscar posibles valores outliers en mi dataset. Para ello, generé una variable para guardar las columnas con un data type flotante o entero, sin contar a la columna de Rings, ya que esa era la variable objetivo. Posteriormente, generé una lista para guardar a los posibles outliers que fuera a encontrar.

Usé el rango intercuartílico de modo que si mi código llegaba a encontrar una instancia con algún valor que se encontrara en:

$$Cota_{inferior} = Q1 - 1.5 * IQR$$

o

$$Cota_{superior} = Q3 + 1.5 * IQR$$

sería tomado como outlier y almacenado en la lista anteriormente mencionada.

Al final, el número de posibles outliers encontrados fue de 151, así que lo siguiente por hacer fue decidir qué iba a hacer con esos outliers. Mi primera opción fue eliminarlos, pero consideré que estaría perdiendo varios datos, aún si sólo se trataba del 3.66% del total de datos (contando los dos registros con Height=0 que eliminé previamente). Luego, me llegó la suposición de que dichos outliers podían tratarse de abulones o muy pequeños o muy grandes. Así que para comprobarlo,

decidí revisar manualmente a los outliers y ver si sus datos eran consistentes entre sí.

Tras la revisión, noté que varios de los datos sí eran extremos, más no inconsistentes. Muchos se trataban o de abulones muy jóvenes cuyos datos eran pequeños debido a su temprana edad, y otros eran abulones muy viejos o grandes que también se alejaban del rango intercuartílico, pero no por eso eran errores.

Sólo hubo un caso que se me hizo extraño, que fue el de un abulón con índice 2051. Este poseía una altura de 1.13mm, el cual era del doble de su diámetro (el cual era de 0.355mm). Esto tendría que ser físicamente incorrecto. Pero a lo mejor podía ser un caso extremo, así que decidí comparar sus datos con los otros 9 abulones más altos de mi dataset. El abulón con índice 2051 fue el más alto de la lista, mientras que el segundo más alto apenas tenía la mitad de su tamaño, pero con un diámetro más grande. Con el resto de los datos fue parecido, de modo que Height tenía no sólo una inconsistencia con su altura, sino que con el resto de sus datos, siendo que el resto de sus características eran por mucho menores a las de los otros 9 abulones más altos. Esa inconsistencia con sus propias características me hizo reconocerlo como un outlier, y decidí eliminarlo de mi dataset. A continuación, muestro la tabla de comparación para que se pueda visualizar mejor la enorme diferencia e inconsistencia de la que hablo:

	Sex	Length	Diameter	Height	Whole	Rings
2051	F	0.455	0.355	1.130	0.5940	8
1417	M	0.705	0.565	0.515	2.2100	10
1763	M	0.775	0.630	0.250	2.7795	12
1428	F	0.815	0.650	0.250	2.2550	14
2179	F	0.595	0.470	0.250	1.2830	14
1528	M	0.725	0.575	0.240	2.2100	13
277	M	0.610	0.500	0.240	1.6420	18
307	M	0.645	0.515	0.240	1.5415	13
2161	F	0.715	0.565	0.240	2.1995	17
2212	F	0.595	0.495	0.235	1.3660	13

Figura 2. Comparación de los abulones más altos

Con ese registro eliminado, me decidí a seguir trabajando con el resto de los abulones considerados posibles outliers para que mi modelo también supiera cómo trabajar con casos extremos, pues, así como mencioné, se trataban de datos extremos pero consistentes. Por eso los dejé. Y con eso terminé la fase de ETL y de limpieza de los datos.

III. Análisis de correlaciones

3.1. One-Hot Encoding de la variable Sex

Antes de dividir el dataset en conjuntos X y Y, al tener un modelo de regresión lineal, no puedo pasarle datos no numéricos al modelo, pero justamente eso es lo que es Sex: una variable no numérica. Al tener 3 categorías distintas (Male, Female e Infant), no podía simplemente asignarle un número del 0-2 o del 1-3 a cada valor, ya que entonces les estaría asignando indirectamente un valor numérico. Por ejemplo, si definiera que Male=1 y Female=2, entonces Female tendría mayor peso que Male y eso no es correcto.

En su lugar, me decidí por transformarlos en valores binarios. Pero claro, tengo tres valores diferentes y no dos. Así que, para resolver esto, luego de convertir la variable Sex en binaria, quité una de las tres variables para evitar que hubiera redundancia entre las variables y el intercepto, de modo que quedaría implícita en las otras dos. En este caso, la variable eliminada, y que quedó como categoría de referencia, fue Sex_F (o sea, las abulones femeninas). Más adelante notarán que es por eso que aparecen Sex_M y Sex_I, pero no Sex_F.

Siendo más específico, al cambiar la variable Sex a binaria, conseguí una tabla parecida a la siguiente:

Sex	Sex_F	Sex_I	Sex_M
F	1	0	0
I	0	1	0
M	0	0	1

Tabla 2. Variable binaria Sex.

Pero, al hacer drop_first=True, eliminé la primera columna de la tabla, que en este caso fue Sex_F. Y, al usar la función .get_dummies() de pandas, transformé las variables categóricas en variables numéricas binarias, de modo que la tabla pasó a tener la siguiente forma:

Sex	Sex_I	Sex_M
F	0	0
I	1	0
M	0	1

Tabla 3. Variable binaria Sex luego de eliminar la primera columna.

Es decir, Sex_F no desapareció del dataset original, sino que se volvió una categoría de referencia, de modo que más

adelante las correlaciones de Sex_M y Sex_I se implementaron respecto a la categoría F.

3.2. Separando el dataset en clase y variables

Ya con todas las variables siendo numéricas, dividí el dataset en conjuntos X (para la clase) y Y (para las variables). Definí a Rings como la clase ya que es la variable objetivo a predecir, mientras que al resto de las características las dejé como variables independientes.

3.3. Cálculo y visualización de las correlaciones

La correlación es la medida estadística que permite saber cómo es que dos o más variables se relacionan entre sí. Esto es importante ya que necesito ver cómo es que las distintas características físicas del abulón se relacionan entre sí y con la variable Rings. Así que, para ello, calculé la matriz de correlaciones entre las variables usando la función .corr() de pandas. Posteriormente, y para facilitar el entendimiento de las correlaciones, creó un Heatmap que graficara los resultados.

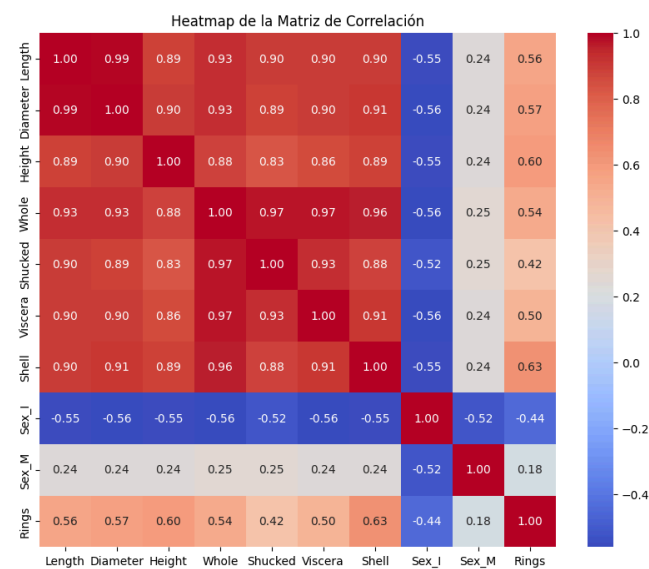


Figura 3. Heatmap de correlaciones

El mapa representa visualmente con colores las correlaciones entre las variables, donde el coeficiente de correlación puede tomar valores entre 1 a -1, siendo que 1.00 es la correlación positiva perfecta, -1.00 es la correlación negativa perfecta y 0.00 significa que no hay correlación.

En este caso, los colores representan lo siguiente:

- **Rojo:** Correlación positiva.
- **Azul:** Correlación negativa.
- **Blanco:** Correlación de 0 o sin correlación.

De modo que mientras más intenso sea el color, mayor es la magnitud de correlación.

Noté que la correlación entre las primeras siete variables (que van de Length hasta Shell) era extremadamente alta, con valores que iban del 83% hasta el 99% de correlación. Es decir, esa información está muy relacionada. Y de hecho eso hace sentido, ya que si un abulón es más grande, va a tener mayor peso, longitud, diámetro y altura. Aunque igualmente eso podría indicar que había multicolinealidad entre las variables independientes, y aunque eso no significa que no pueda usar la regresión lineal múltiple, sí va ser algo que afecte más adelante en los pesos individuales del modelo.

En el caso de Sex, podemos notar que Sex_I tuvo una asociación moderadamente negativa con el resto de las variables, mientras que Sex_M tiene una asociación más débil pero positiva. Y recordemos que todo esto es tomando implícitamente a Sex_F como categoría de referencia. O sea, ahora mismo la variable que estamos buscando predecir es la edad, y definimos al sexo femenino como nuestra categoría de referencia, por lo que se están comparando las variables de los abulones machos e infantes con las variables de las hembras. Por eso los abulones infantes tienen valores negativos, por que al ser más jóvenes, tienen menos edad y por ende son mucho más pequeños que el resto de las bulones hembras, mientras que los machos resultaron ser, en promedio, más viejos y grandes que las hembras, por eso sus valores son positivos pero más pequeños, por que la diferencia entre los macho y las hembras es pequeña.

En conclusión, todas las características físicas tienen correlaciones extremadamente fuertes que van aproximadamente de $0.83 \leq r \leq 0.99$; mientras que la correlación entre las variables físicas y Rings es más moderada, que va aproximadamente de $0.42 \leq r \leq 0.63$.

IV. Preparación de los datos para Machine Learning

4.1. División de los datos en conjuntos Train y Test

En el dataset actual, luego de la limpieza inicial, quedaron en total 4,174 instancias de abulones, pero la cuestión es que no podía a mostrarle todos esos abulones al modelo ya que entonces no habría con qué probarlo, y solamente estaría memorizando la información, no aprendiendo a predecir el número de anillos. Así que, para evitar ese problema, dividí mis

datos de la siguiente forma: 80% van a ser dedicados a entrenar el modelo y el otro 20% van a ser para probar y evaluar el modelo. Existen otras formas de dividir los datos, pero escogí esta debido a su equilibrio a la hora de dividir los datos.

Para lograr dividir los datos sin usar librería o frameworks de Machine Learning, mezclé los datos usando la librería Random (la cual no es de ML) para que la división de los datos fuera, justamente, aleatoria, y definí una semilla para siempre tener la misma semilla de datos, ya que no quería que con cada prueba mis resultados cambiaran. definí la proporción de entrenamiento de 0.8 y lo multipliqué por el conjunto de datos ya mezclados, con lo cual pude obtener el 80% de los datos para el entrenamiento, y el resto de los datos sobrantes los recopilé usando slicing, recuperando así el último 20% de los datos para pruebas. Finalmente, dividí mis conjuntos de datos X y Y usando los índices de entrenamiento y prueba, de modo que todos los datos en entrenamiento tuvieran los mismos índices en el mismo orden, y lo mismo para el conjunto de pruebas (porque si no, entonces estaría combinando datos de diferentes índices y eso sería desastroso para el modelo).

Aunque en mi código no lo incluyo porque dejó de ser necesario, sí hice una comprobación para revisar si habían registros repetidos entre los conjuntos de entrenamiento y prueba. Igualmente revisé que estuvieran todos los 4174 registros entre los dos conjuntos. Luego de esta comprobación, comprobé que efectivamente no se repetían índices de datos entre conjuntos y que tenía todos y cada uno de mis registros.

```
print(
    "¿Hay índices repetidos?",
    len(set(train_indices) & set(test_indices)) > 0
)

¿Hay índices repetidos? False
```

Figura 4. Comprobación de índices repetidos

```
print(
    "¿Están todos los registros incluidos?",
    len(set(train_indices) | set(test_indices)) == len(df_x_encoded)
)

¿Están todos los registros incluidos? True
```

Figura 5. Comprobación de que estén todos los registros

4.2. Estandarización de las variables

Antes de pasar al modelo de regresión lineal múltiple, tuve que hacer unos últimos cambios ya que la escala de mis datos era muy diferente entre sí. Los registros de las variables como Length iban de de

0.075mm a 0.815mm, los de Whole de 0.002 a 2.8 gramos y las columnas de Sex iban de 0 a 1 sin intermedios. Tenía valores muy bajos, otros relativamente altos y otros eran simplemente binarios. Así que tuve que estandarizar la escala de mis datos para facilitar la optimización por parte del gradiente descendente.

El objetivo era transformar cada variable usando la fórmula de estandarización:

$$z = (x - \mu) / \sigma$$

Donde:

- x = valor original
- μ = Media de la variable
- σ = desviación estándar
- z = valor estandarizado

Pero esto no lo podía hacer directamente con `df_x_enconded`, ya que eso haría que estuviera usando información de los datos de prueba para preparar los datos de entrenamiento. Es por eso que calculé la media y la desviación estándar usando solamente `x_train` y usé esos mismos datos para `x_test` también, para estandarizar ambos conjuntos y que el modelo no pudiera ver el contenido del conjunto de pruebas.

Una vez estandarizados ambos conjuntos, los datos obtuvieron una media de aproximadamente 0 (ya que los valores reales fueron muy cercanos a cero) y con una desviación estándar de 1.

```
Media de X_train_scaled:
Length      -1.244886e-16
Diameter    -2.330172e-16
Height       2.298251e-16
Whole        1.851369e-16
Shucked      1.915210e-17
Viscera      -6.596833e-17
Shell        -7.660838e-17
Sex_I         3.511217e-17
Sex_M         8.512042e-18
dtype: float64

Desviación estándar de X_train_scaled:
Length      1.0
Diameter     1.0
Height       1.0
Whole        1.0
Shucked      1.0
Viscera      1.0
Shell        1.0
Sex_I        1.0
Sex_M        1.0
dtype: float64
```

Figura 6. Resultados de la estandarización

A continuación, voy a explicar el por qué de estos resultados.

La media ≈ 0 es por que al restarle la media a cada valor de la variable al hacer $x - \mu$, desplacé toda la distribución de datos de modo que dicha distribución quedó centrada cerca de 0. Por eso los valores de la media son tan pequeños, no están exactamente en cero, pero sí moví la distribución para que estuviera centrada lo más cerca del 0 posible. Y luego, al dividir el resultado entre la desviación estándar (σ), cambié la escala de la variable y hice que sus unidades se mostraran en unidades de desviación estándar, de ahí el $\sigma = 1$. Esto para que las variables quedaran en una escala comparable cuando llegara el momento de trabajar con el gradiente descendente..

V. Implementación del modelo

5.1. Definición del modelo

Para poder predecir el número de anillos de un abulón, ya dije que trabajaría con la regresión lineal múltiple, de modo que iba a estar trabajando con la siguiente fórmula:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \dots + \beta_9 x_9$$

donde:

- $\hat{y}$ = predicción
- x = característica
- β_0 = intercepto/bias
- β_{1-9} = el resto de las 9 variables

Para mi regresión lineal múltiple trabajé con 9 características y, por ende, con 9 pesos más un intercepto, formando un total de 10 parámetros.

Ahora, no tenía ni conocía los pesos para β_{0-9} (incluyendo el bias), así que el trabajo del gradiente descendente fue encontrar justamente los valores para esos pesos de modo que las predicciones fueran lo más cercanas posible a los Rings reales.

5.2. Inicialización de los parámetros

Para calcular los pesos podría haber tenido 9 variables diferentes que fueran de $\beta_1 = 0$ a $\beta_9 = 0$ pero, en vez de hacer eso, decidí ahorrarme tiempo creando una matriz que ya contuviese de β_1 - β_9 , de modo que el bias (β_0) quede separado.

Como aún no sabía cuales eran los mejores pesos (bias incluido), asumí que todos los β eran iguales a 0. Es decir: $\beta_1=\beta_2=\dots=\beta_9=0$ y $\beta_0=0$.

5.3. Creación de la función de predicción

Realicé mi primera predicción de prueba. Recordemos que para ese momento mi conjunto de entrenamiento tenía una dimensión de 3339x9, de los 3339 registros de mi dataset luego de dividir los datos en conjuntos de train y test, y de las 9 columnas con las características del abulón; por lo que también tenía 9 pesos. Entonces, para generar la primera predicción, podría hacer una multiplicación entre matrices de la siguiente manera:

$$\hat{y} = XW$$

donde:

- X: son mis valores del conjunto de entrenamiento
- W: son los pesos

Lo que obtuve fue el producto entre las matrices de X y de pesos, de modo que hice prácticamente la siguiente operación:

$$\hat{y} = x_1w_1 + x_2w_2 + \dots + x_9w_9 + bias$$

Así que, con eso implementado, realicé mis primeras predicciones donde, como inicialmente tanto los pesos como el bias son 0, mis predicciones también dieron 0 de resultado.

5.4. Creación de la función de costo MSE

Ahora viene la duda del millón, y es: ¿cómo podía saber qué tan malas fueron las predicciones que hice? Ahora tenía las predicciones $\hat{y}$ y los valores reales de 'y'. Muy probablemente los valores reales de 'y' no iban a ser 0, así que probablemente el modelo actual era pésimo, por ende, necesitaba medir ese error numéricamente. Para ello, implementé la función de costo MSE:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Donde:

- y_i = valor real

- $\hat{y}_i$ = predicción
- $y_i - \hat{y}_i$ = error
- $(y_i - \hat{y}_i)^2$ = error al cuadrado
- n = número de observaciones

Aclaración: La razón por la que elevamos el error al cuadrado es para evitar que se anulen los valores positivos con los negativos. El objetivo es que el MSE sea lo más pequeño posible, ya que eso implica un menor error y, por ende, que la predicción está funcionando.

Luego de implementada la función de costo, calculé el MSE de mi primera predicción, a lo que obtuve que MSE=110.3354 Como el modelo predijo 0 para todo, obtuve un error bastante grande. Esto ya me decía qué tan mal estaba mi modelo, más no me decía cómo debía corregir los pesos. No me decía si debía aumentar o disminuir algún peso. Para lograr actualizar los pesos necesitaba implementar el Gradiente Descendente, y eso es justo lo que vamos a ver a continuación.

5.5. Entendiendo la derivada del Gradiente Descendente

Ya conseguí el primer valor de MSE, el cual es bastante malo. Ahora el objetivo era buscar valores para los pesos que minimizaran el valor del MSE. Por eso es que debía usar la derivada: para buscar dichos valores en los pesos que minimizaran lo más posible el valor de MSE.

Por poner un ejemplo, podemos imaginar que nuestra función nos posiciona en una colina. El objetivo era llegar a la base de dicha colina, que es donde estaría el MSE más pequeño, donde para eso probaría dar pasos en diferentes direcciones para poder ir bajando o subiendo poco a poco según lo que requiera para llegar a la base. Digamos que si estaba en una posición 5, debía bajar la colina; y si estaba en una posición -5, es porque estaba en una depresión y necesitaba subir. Esto de forma iterativa hasta encontrar la base de la colina.

Lo mismo se iba a hacer aquí. El objetivo era actualizar los pesos y sus direcciones (para ver si sus valores debían subir o bajar) y calcular su MSE de forma iterativa hasta encontrar aquellos valores que minimizaran lo más posible el MSE. Cabe aclarar que sería imposible llegar a un error de 0 (eso significaría que mi modelo se sobre ajustó a los datos), pero eso no

significaba que no pudiera intentar aproximarme lo más posible.

En términos matemáticos, es lo mismo que hacer esto:

$$\frac{\partial MSE}{\partial w_1}$$

Esto representa cómo es el cambio en MSE cuando se modifica w_1 . Sería hacer eso para cada peso, incluyendo el bias:

$$\frac{\partial MSE}{\partial w_1} \dots \frac{\partial MSE}{\partial w_9}$$

Con eso claro, el siguiente paso era derivar la gradiente. Actualmente tenía las siguientes fórmulas:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\hat{y}_i = b + \sum_{j=1}^9 w_j \cdot x_{ij}$$

Y el objetivo era obtener:

$$\frac{\partial MSE}{\partial w_j}$$

Para eso hice regla de la cadena, lo cual me llevó a las fórmulas de las derivadas de MSE con respecto a w_j y de MSE con respecto al bias. A continuación, describo el proceso matemático que realicé para llegar a dichas fórmulas.

1) Tenemos que el término que depende de w_j es $\hat{y}_i$, así que usé la regla de la cadena:

$$\frac{\partial MSE}{\partial w_j} = \frac{\partial MSE}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial w_j}$$

2) Derivé MSE con respecto a $\hat{y}_i$:

$$\frac{\partial MSE}{\partial \hat{y}_i} = \frac{1}{n} \cdot 2(\hat{y}_i - y_i) = \frac{2}{n} (\hat{y}_i - y_i)$$

Aclaración: No se toma en cuenta el símbolo Σ ya que esto se hace para cada término de la sumatoria, o sea, para valor de i .

3) Derivé $\hat{y}_i$ con respecto a w_j :

$$\frac{\partial \hat{y}_i}{\partial w_j} = 0 + 1 \cdot x_{ij} = x_{ij}$$

4) Multipliqué los resultados:

$$\frac{\partial MSE}{\partial w_j} = \frac{2}{n} \sum_{i=1}^n x_{ij} \cdot (\hat{y}_i - y_i)$$

5) Obtuve la derivada con respecto al bias, donde como es sesgo b afecta a todas las predicciones $\hat{y}_i$ por igual, entonces tengo que:

$$\frac{\partial \hat{y}_i}{\partial b} = 1$$

Por ende:

$$\frac{\partial MSE}{\partial b} = - \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

Esto se da porque como el sesgo b no depende de i ni de ninguna características x_{ij} , al cambiar b , todas las predicciones cambian la misma cantidad. Por ejemplo, si b aumenta en 1, entonces cada $\hat{y}_i$ aumentará en 1 también. Por eso obtenemos $\partial \hat{y}_i / \partial b = 1$.

Aclaración: El signo “-” en $\partial MSE / \partial b$ aparece al reescribir el término $(\hat{y}_i - y_i)$ a $-(y_i - \hat{y}_i)$. Esto se hace para expresar el gradiente en la dirección del descenso del error, que es la que se usa en el algoritmo del Gradiente Descendente.

Explicando mejor qué significa cada parte, el gradiente de un peso nos dice que tenemos $y_i - \hat{y}_i$, es decir, *error = valor real - valor predicción*. El x_{ij} es el valor que tenía la determinada característica del abulón. Así que al hacer $x_{ij} \cdot (y_i - \hat{y}_i)$, lo que estábamos diciendo es qué tan grande es el error x cuándo está involucrada esa característica en esa observación. Y eso se hizo para todos los abulones y así se obtuvo un valor que indicaba cómo es que afectaba modificar ese peso al MSE.

Es aquí donde entra la gradiente, ya que como tenía 9 pesos, iba a terminar teniendo nueve derivadas, y ese conjunto de derivadas iba a ser el gradiente. El gradiente se iba a utilizar para actualizar el valor de los pesos usando la siguiente fórmula:

$$w_j \leftarrow w_j - \alpha \left(\frac{\partial \text{MSE}}{\partial w_j} \right) \text{ para los pesos}$$

$$b \leftarrow b - \alpha \left(\frac{\partial \text{MSE}}{\partial b} \right) \text{ para el bias}$$

Donde α es el **learning rate**. Explicando más el concepto de learning rate, este es el hiper parámetro que definirá cuál será el tamaño de los pasos del modelo de Machine Learning al actualizar los pesos una vez se inicie su entrenamiento.

Importante: El Learning rate se debe escoger con cuidado, ya que un learning rate muy pequeño va a dar la respuesta correcta, pero va a tardar mucho en darla; mientras que un learning rate muy alto va a dar saltos demasiado grandes que compliquen encontrar los mejores valores de los pesos para disminuir el valor del MSE.

5.6. Implementación del gradiente descendente

Así que, con eso dicho, procedí a desarrollar la función que calculara el gradiente, a lo que luego de correrlo obtuve los siguientes resultados:

```
Gradiente de los pesos:
[-3.58724163 -3.70128949 -3.87732678 -3.45721462 -2.6847121 -3.23277553
-4.04301417 2.81159865 -1.13477714]

Gradiente del bias:
-19.982030548068284
```

Figura 7. Cálculo del primer gradiente

Como se puede observar, conseguí 10 gradientes debido a que tengo 9 pesos diferentes y un bias.

Definí un learning rate de 0.01 y usé los gradientes que obtuve previamente para actualizar los pesos, a lo que obtuve el siguiente resultado:

```
Nuevos pesos:
[ 0.03587242 0.03701289 0.03877327 0.03457215 0.02684712 0.03232776
 0.04043014 -0.02811599 0.01134777]

Nuevo bias:
0.19982030548068286
```

Figura 8. Nuevos pesos del modelo

Con esto mi modelo ya dió su primer paso de aprendizaje, así que ahora debo comprobar que realmente haya aprendido revisando que el MSE nuevo sea menor al MSE viejo. Luego de comparar, vi que el nuevo MSE era de 105.4808, mientras que el viejo, cuando todos los pesos tenían un valor de 0, era de 110.3354. Eso significaba que la fórmula de gradiente funcionaba correctamente, así que procedí a realizar el mismo proceso pero de forma iterativa y automática.

Pero antes de eso, tuve que definir cuántas iteraciones se iban a hacer. No podía simplemente dejar que el modelo iterara eternamente y esperar a que llegara a un MSE cercano a cero luego de una cantidad indefinida pero alta de iteraciones. En lugar de eso, definí la condición de que el máximo de iteraciones (o epochs) iban a ser 1000. Esto garantizaría que eventualmente las iteraciones fueran a terminar, sin importar el valor que tuviese el MSE. Así que me mantuve con las 1000 iteraciones (epochs) y con un learning rate de 0.01.

Solo que, antes de iniciar con la regresión, previamente ya había hecho una actualización manual para comprobar que mi función de gradiente funcionara, como ya pudieron leer anteriormente. Así que lo que hice fue reiniciar los pesos para que sus valores fueran nuevamente de 0, de modo que el entrenamiento se pudiera desarrollar de la forma más limpia posible.

Una vez construido el ciclo para actualizar los pesos, agregué un output que me devolviera el valor del MSE cada 100 epochs para así documentar cómo es que descendía el error. Luego de los 1000 epochs, conseguí los siguientes resultados:

```
Epoch 0: MSE = 110.3354
Epoch 100: MSE = 7.8379
Epoch 200: MSE = 5.6426
Epoch 300: MSE = 5.3601
Epoch 400: MSE = 5.2150
Epoch 500: MSE = 5.1277
Epoch 600: MSE = 5.0729
Epoch 700: MSE = 5.0371
Epoch 800: MSE = 5.0130
Epoch 900: MSE = 4.9961
```

Figura 9. Descenso del MSE a lo largo de las épocas

Así que, en general, lo que se hizo en este ciclo fue empezar con pesos iniciales de 0, donde obtuve la predicción y el MSE, el cual dijo qué tan equivocada estaba la predicción. Luego de eso se calculó el gradiente de MSE (∇MSE) que diría cómo cambiar los parámetros de los pesos, a lo que finalmente se actualizarían dichos pesos y el ciclo volvería a empezar hasta que se cumpliera la condición que ya mencioné anteriormente, que sería llegara las 1000 iteraciones.

Ya con la regresión lineal múltiple con gradiente descendente finalizada, y el MSE actualizado, decidí graficar los resultados para analizar la tendencia descendente del modelo.

Aclaración: No buscaba que necesariamente la gráfica o el valor de MSE llegara a 0. Eso significaría que las predicciones de mi modelo son perfectas, lo cual sería muy poco probable a menos que mi modelo se adaptara para predecir a la perfección sólo los valores de entrenamiento (o sea, que se haya generado un overfitting). El objetivo es que el MSE disminuya lo más posible y que las mejoras sean cada vez más pequeñas.

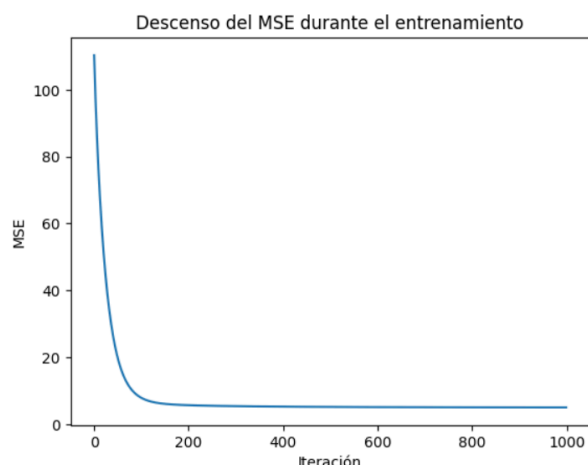


Figura 10. Gráfica del descenso del MSE a lo largo de las épocas

Efectivamente, mi modelo de regresión lineal con gradiente descendente disminuyó el MSE y permitió que el modelo aprendiera correctamente. Lo que más se puede destacar de estos resultados es el increíble decremento del error en las primeras 100 iteraciones, pasando de un error de 110.33 a uno de 7.83. De ahí en adelante, el error siguió disminuyendo pero a un nivel mucho menor, de modo que el MSE final que conseguí para mi conjunto de pruebas fue de aproximadamente 4.99.

En conclusión, ya había comprobado que mi modelo aprendió. Ahora venía la parte más importante: ver si mi modelo funcionaba para predecir el resto del 20% de los registros que no vió durante su etapa de entrenamiento.

5.7. Generación de predicciones al conjunto de pruebas

Como ya mencioné, ahora tocaba revisar qué tan bien funcionaba el modelo con registros que nunca había visto y que no utilizó durante su entrenamiento.

Como ya anteriormente definí la función de predicción (la función predict), pues la utilicé pero para el conjunto de pruebas.

Aclaración: Utilicé el conjunto de pruebas escalado y no el conjunto de prueba original para mantener la versión del dataset con los datos estandarizados, de modo que se mantuviera la misma escala que usó el modelo para su entrenamiento.

Ya con la regresión aplicada a los datos de prueba, decidí generar una tabla que comparara las primeras 10 predicciones del modelo con los valores reales de Rings de dichos registros, a lo que obtuve la siguiente tabla:

Valores Reales	Predicciones
9	11.43
17	11.85
11	12.53
8	7.23
12	11.63
8	8.96
9	12.08
11	9.47
8	9.92
8	9.00

Tabla 4. Comparación de valores reales con predicciones

VI. Resultados

6.1. Predicciones contra valores reales

Una vez comparados los resultados de la predicción de mi modelo con los reales, se puede notar que algunas predicciones son mejores que otras. Para saber exactamente cuál fue el MSE, utilicé la función del mismo nombre que definí anteriormente (función `mse()`) para calcular el error cuadrático medio de mi predicción y utilicé los resultados para generar una gráfica en para comparar todos los registros reales con los predichos, de modo que conseguí el siguiente resultado:

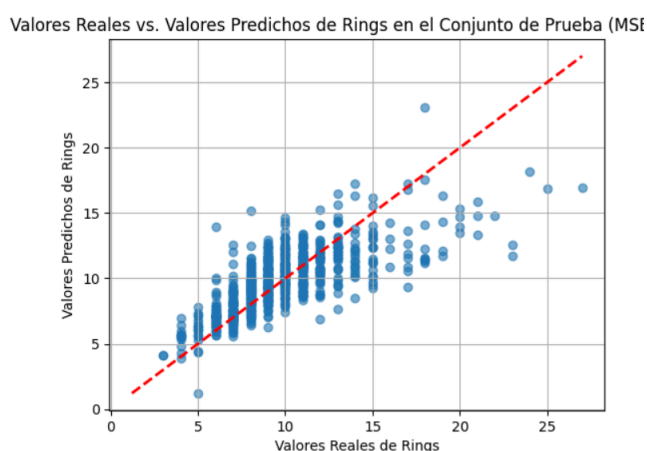


Figura 11. Comparación entre valores reales y predichos de Rings

Al final, el MSE de las predicciones de mi conjunto de pruebas fue de 4.606, lo que significa que, en promedio, el cuadrado de la diferencia entre mis valores reales y los predichos por mi modelos fue de 4.606. O, en términos de RMSE, la diferencia entre la cantidad de anillos predicha y real fue de aproximadamente 2.15 anillos.

Si lo comparo con el MSE de mi conjunto de entrenamiento que fue aproximadamente de 4.98, el MSE de mi conjunto de prueba es similar al que obtuve durante el entrenamiento del modelo. Eso significa que mi modelo tiene un rendimiento similar cuando trabaja con datos de entrenamiento que cuando trabaja con datos nuevos, eso significa que su capacidad para generalizar es bastante razonable.

Aclaración: Algo importante a notar es que el modelo devolvió predicciones con valores flotantes (o sea, con punto decimal) para los anillos cuando se supone que estos son enteros. Esto no es que esté mal o

sea un error, simplemente se trata de una estimación que no es perfecta y que usa esos valores decimales para poder calcular el MSE.

Referencias bibliográficas

- [1] Ashok Kumar Palivela (2021). “Predicción de la edad del abulón”. Recuperado de: <http://kaggle.com/code/ashokkumarpalivela/abalone-age-prediction>
- [2] Harshitha Indumathi (2021). “Predicción de la edad del abulón”. Recuperado de: <https://harshithaindumathi.medium.com/abalone-age-prediction-607ecda3fa3b>
- [3] Brayan Buitrago (2020). “Aprendizaje automático — Modelos de Regresión I”. Recuperado de: <https://medium.com/iwannabedatadriven/machine-learning-modelos-de-regresi%C3%B3n-i-d293ae235e9a>
- [4] Khan Academy (S.F.). “Descenso de gradiente”. Recuperado de: <https://es.khanacademy.org/math/multivariable-calculus/applications-of-multivariable-derivatives/optimizing-multivariable-functions/a/what-is-gradient-descent>
- [5] AWS (2022). “¿Qué es extracción, transformación y carga (ETL)?”. Recuperado de: <https://aws.amazon.com/es/what-is/etl/>
- [6] Informatica (S.F.). “¿Qué es ETL (extracción, transformación y carga)?”. Recuperado de: <https://www.informatica.com/resources/articles/what-is-etl.html>
- [7] GeeksForGeeks (2026). “Codificación One-Hot en Aprendizaje Automático”. Recuperado de: <https://www.geeksforgeeks.org/machine-learning/ml-one-hot-encoding/>
- [8] Vinod Chugani (2025). “Codificación One-Hot: Entendiendo lo “caliente” en los datos”. Recuperado de: <https://machinelearningmastery.com/one-hot-encoding-understanding-the-hot-in-data/>
- [9] Michael DelSole (2018). “¿Qué es la codificación One Hot y cómo se realiza?”. Recuperado de: <https://medium.com/@michaeldelsole/what-is-one-hot-encoding-and-how-to-do-it-f0ae272f1179>
- [10] jmp Statistical Discovery (S.F.). “Correlación”. Recuperado de: <https://medium.com/@michaeldelsole/what-is-one-hot-encoding-and-how-to-do-it-f0ae272f1179>
- [11] Máxima formación (2020). “¿Qué es la correlación estadística y cómo interpretarla?”. Recuperado de: <https://www.maximaformacion.es/blog/que-es-la-correlacion-estadistica-y-como-interpretarla>
- [12] Mirko Stojiljković (2019). “NumPy, SciPy y pandas: correlación con Python”. Recuperado de:

<https://realpython.com/numpy-scipy-pandas-correlation-python/>

- [13] Eda Kavlakoglu y Jacob Murel Ph.D. (S.F.). “¿Qué es la multicolinealidad?”. Recuperado de:
<https://www.ibm.com/mx-es/think/topics/multicollinearity>
- [14] Máxima Formación (2022). “¿Qué es la multicolinealidad y por qué es un problema?”. Recuperado de:
<https://www.maximaformacion.es/blog/que-es-la-multicolinealidad-y-por-que-es-un-problema>
- [15] Google for developers (S.F.). “Regresión lineal: Hiperparámetros”. Recuperado de:
<https://developers.google.com/machine-learning/crash-course/linear-regression/hyperparameters?hl=es-419>
- [16] Priyansh Soni (2024). “Hiperparámetros en el aprendizaje automático explicados”. Recuperado de:
<https://www.blog.trainindata.com/hyperparameters-in-machine-learning/>